VITA

Real-Time AI Health Coach

MVP Technical Design Document — v1.0

Optimized for Claude Code implementation

April 2026

Table of Contents

1. Product Overview

Vita is a Progressive Web App (PWA) that acts as a real-time, context-aware personal health coach. It eliminates the guesswork from staying healthy by combining four data input channels — voice, photo, calendar, and manual text — with a persistent AI brain that always knows your full situation before giving advice.

Unlike generic nutrition apps that react to what you log, Vita is proactive. It knows you have a flight in 90 minutes, that you've only had coffee today, and that you have a client dinner tonight. It coaches accordingly.

> **Core philosophy:** Every Claude API call is assembled with full situational context — food log, time of day, calendar events, and user goals — so advice is always personally relevant, never generic.

1.1 Key Differentiators

- Context-first AI: the brain layer bundles all state into one rich prompt before every call
- Four input modalities: voice (Web Speech API), photo (camera → Claude Vision), calendar (Google OAuth), manual
- PWA-native: installable, full-screen, push notifications, Action Button shortcut — feels like a native iOS app
- Smart notifications: fired by calendar events…
- Structured outputs: Claude returns JSON for every meal parse so UI renders macros deterministically

1.2 What This Document Is

This document is the single source of truth for building Vita's MVP. It is written to be consumed directly by Claude Code with no ambiguity. Every file, every API contract, every edge case, and every small implementation detail is specified here. Read it top to bottom before writing any code.

2. System Architecture

2.1 High-Level Layers

The system has three logical layers that run in the browser with no backend server required for the MVP:

Layer	Responsibility	Key Technologies
Input Layer	Capture data from 4 sources and normalize to text/base64	Web Speech API, MediaStream API, Google Calendar API, HTML forms
Brain Layer	Assemble context, call Claude, parse response	Anthropic Messages API (claude-sonnet-4-20250514), JSON parsing
Output Layer	Render log, coach chat, notifications	DOM, Service Worker Push, IndexedDB, CSS animations

2.2 Data Flow — Per Interaction

1. User triggers an input (taps mic, opens camera, types, or app auto-fires from calendar)
2. Input module normalizes data: voice → transcript string; photo → base64 + mime; calendar → JSON event array; text → raw string
3. Context builder (contextBuilder.js) assembles a single system prompt containing: current time, user profile/goals, today's food log so far, pending calendar events
4. Claude API called with assembled context + user message/image
5. Response parser (parseClaudeResponse.js) extracts structured JSON for meals OR plain text for coach messages
6. UI updates: food log appended, macros recalculated, coach message displayed
7. IndexedDB persisted immediately (logStore.js) — survives page refresh
8. Service Worker checks calendar queue — fires push notification if threshold met

2.3 File Structure

All source lives in a single flat /src directory to minimize import complexity. No build step required for MVP — vanilla ES modules via <script type='module'>.

```
/vita-mvp
  /src
    app.js                ← main entry, initialises all modules
    contextBuilder.js     ← assembles Claude system prompt (CORE)
    claudeClient.js       ← wrapper around fetch() for Anthropic API
    parseClaudeResponse.js ← extracts JSON meals + plain coach text
    voiceInput.js         ← Web Speech API handler
    photoInput.js         ← camera + base64 encoder
    calendarClient.js     ← Google Calendar OAuth + event fetcher
```

```
  logStore.js          ← IndexedDB read/write for food log + goals
  notificationManager.js ← Service Worker registration + push logic
  ui.js                ← all DOM manipulation (single file)
  macroEngine.js       ← local macro summation, no AI needed
/styles
  app.css              ← single stylesheet, CSS variables, mobile-first
index.html             ← shell, loads app.js as module
manifest.json          ← PWA manifest
sw.js                  ← Service Worker (push + cache)
.env                   ← ANTHROPIC_API_KEY, GOOGLE_CLIENT_ID (not committed)
```

Note: No React, no bundler, no build step. Vanilla JS modules load natively in Safari/Chrome on iOS. This keeps the project deployable by dropping files in any static host (Netlify, Vercel, GitHub Pages).

3. Brain Layer — Context Builder (The Most Important File)

contextBuilder.js is the heart of Vita. Its only job is to assemble the richest possible system prompt before every Claude call. This is what makes advice feel personal rather than generic. Build this file first and test it thoroughly before building any UI.

3.1 System Prompt Structure

The assembled system prompt has the following sections in order. Each section is separated by a blank line. Variable placeholders shown in {brackets}.

```
You are Aria, a personal health coach inside the Vita app. Be warm, specific, and
brief.
Never give generic advice — always reference the user's actual data below.

## User Profile
Name: {name} | Age: {age} | Sex: {sex}
Goal: {primaryGoal}  (e.g. 'lose 0.5 lbs/week' or 'build muscle')
Daily targets: {calGoal} kcal | {proteinGoal}g protein | {carbGoal}g carbs |
{fatGoal}g fat | {fiberGoal}g fiber | {waterGoal}L water

## Current Time
{currentTime} on {currentDate} ({dayOfWeek})

## Today's Food Log ({totalKcal} kcal so far)
{foodLogSummary}
(empty if nothing logged yet)

## Today's Calendar Events
{calendarEvents}
(empty if calendar not connected or no events)

## Hydration
{waterLoggedL}L of {waterGoal}L consumed today

## Instructions
- For meal logs: respond ONLY with valid JSON in the format specified.
- For coach questions: respond in plain text, 2-4 sentences max.
- For notifications: be urgent but not alarming.
- Always reference specific numbers from the user's data, never speak in generalities.
```

3.2 foodLogSummary Format

The food log is summarised as a compact multi-line string. Each line is one meal entry:

```
8:12am — Greek yogurt, granola, blueberries → 380 kcal | P:12g C:58g F:9g
```

```
12:45pm — Grilled chicken salad, olive oil dressing → 520 kcal | P:42g C:18g F:22g
TOTAL SO FAR: 900 kcal | P:54g C:76g F:31g
```

This format is intentionally terse. Claude doesn't need full nutritional databases — it knows approximate macros from training data. The summary should never exceed 20 lines; if it does, collapse earlier meals into a running total.

3.3 calendarEvents Format

Calendar events are formatted as a brief list. Only include events within the next 12 hours that are medically or nutritionally relevant. Filter out recurring status meetings, etc. unless the user has a workout keyword.

```
09:00 — Team standup (30 min) — no action needed
11:30 — Flight to New York (boarding 11:00) — HIGH PRIORITY: only 45 min to eat
19:30 — Client dinner at Nobu — plan lighter lunch; calorie budget for dinner
06:30 tomorrow — Morning run (60 min, from past pattern) — remind tonight
```

Implementation detail: Calendar event relevance scoring: events with keywords [flight, board, gate, run, gym, workout, race, marathon, dinner, lunch, meeting over 2hr] get HIGH priority and must appear. Events without these keywords are LOW priority and omitted unless fewer than 3 events exist.

3.4 contextBuilder.js — Full Spec

Export a single async function: buildContext(). It reads from logStore, calendarClient, and the user profile, then returns { systemPrompt: string, todayLog: array }.

```
export async function buildContext() {
  const profile = await logStore.getProfile();
  const todayLog = await logStore.getTodayLog();
  const water = await logStore.getTodayWater();
  const events = await calendarClient.getTodayEvents(); // [] if not connected
  const summary = formatFoodLog(todayLog);
  const calText = formatCalendar(events);
  const now = new Date();
  const systemPrompt = TEMPLATE
    .replace('{name}', profile.name)
    ... (all replacements)
  return { systemPrompt, todayLog };
}
```

4. Claude API Client (claudeClient.js)

4.1 Model & Parameters

Parameter	Value	Reason
model	claude-sonnet-4-20250514	Best cost/performance for real-time coaching
max_tokens	1000	Sufficient for JSON meal parse or coach response; prevents runaway spend
temperature	not set (default)	Default is optimal; don't override for factual nutrition tasks
stream	false for MVP	Simpler to parse; add streaming in v2 for coach chat feel

4.2 Request Types

There are three distinct Claude call types. Each has a different user message structure:

Type 1 — Meal Log (text)

Triggered by: voice transcript or manual text that contains food.

```
User message:
"Log this meal: [TRANSCRIPT/TEXT]

Respond ONLY with this JSON, no other text:
{
  \"meal\": \"[human-readable name]\",
  \"time\": \"[HH:MM]\",
  \"kcal\": [number],
  \"protein_g\": [number],
  \"carbs_g\": [number],
  \"fat_g\": [number],
  \"fiber_g\": [number],
  \"coaching_note\": \"[1 sentence specific to user's goals/context]\"
}"
```

Type 2 — Meal Log (photo)

Triggered by: camera capture. The message is a multimodal content array.

```
User message: content array with image (base64) + text instruction:
"Identify what's in this photo and log it as a meal.
Respond ONLY with the JSON format above."
```

Type 3 — Coach Question

Triggered by: manual chat input or chip tap in the coach drawer.

```
User message: [raw user question]
No JSON instruction — Claude responds in plain text (2-4 sentences).
```

4.3 Request Detection Logic

Before calling Claude, classify the input type using classifyInput(text):

```
const FOOD_KEYWORDS = ['ate', 'had', 'drank', 'ate', 'breakfast', 'lunch', 'dinner,
  'snack', 'coffee', 'protein', 'bar', 'shake', 'pizza', 'salad', ...];

function classifyInput(text) {
  const lower = text.toLowerCase();
  if (FOOD_KEYWORDS.some(k => lower.includes(k))) return 'meal_text';
  return 'coach_question';
}
```

Photos always go to Type 2. Voice output goes through classifyInput. Manual text goes through classifyInput.

4.4 Response Parsing (parseClaudeResponse.js)

This module handles the two response shapes reliably. Never assume clean JSON — Claude occasionally wraps it in markdown fences.

```
export function parseClaudeResponse(responseText, type) {
  if (type === 'coach_question') {
    return { type: 'coach', text: responseText.trim() };
  }
  // Strip markdown fences if present
  const cleaned = responseText
    .replace(/^```json\n?/, '')
    .replace(/^```\n?/, '')
    .replace(/\n?```$/, '')
    .trim();
  try {
    const meal = JSON.parse(cleaned);
    // Validate required fields
    const required = ['meal','kcal','protein_g','carbs_g','fat_g'];
    for (const f of required) {
      if (meal[f] === undefined) throw new Error('missing ' + f);
    }
    return { type: 'meal', meal };
  } catch (e) {
    // Fallback: treat as coach text
    return { type: 'coach', text: responseText.trim() };
  }
}
```

4.5 API Key Handling

Security note: For MVP/demo, the key lives in a .env file and is injected at load time via a tiny inline script that reads window._vitaConfig. Do NOT hardcode the key in JS files. For production, proxy through a Cloudflare Worker or Vercel Edge Function.

```
// index.html — before loading app.js
<script>
  window._vitaConfig = {
    anthropicKey: '__ANTHROPIC_KEY__',  // replaced by deploy script
    googleClientId: '__GOOGLE_CLIENT_ID__'
  };
</script>
```

5. Input Layer

5.1 Voice Input (voiceInput.js)

Uses the Web Speech API (SpeechRecognition / webkitSpeechRecognition). Works in Safari on iOS 17+ and Chrome on Android/desktop. Not available in Firefox — show a polite fallback.

Initialization

```
const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
if (!SpeechRecognition) { ui.showVoiceFallback(); return; }

const recognition = new SpeechRecognition();
recognition.continuous = false;    // one utterance per press
recognition.interimResults = true; // show live transcript
recognition.lang = 'en-US';
recognition.maxAlternatives = 1;
```

Events to handle

- onresult: update live transcript display with event.results[0][0].transcript
- onend: if final transcript non-empty, call claudeClient.logMeal(transcript)
- onerror (network): show 'Voice unavailable — type instead'
- onerror (not-allowed): show 'Microphone permission denied — go to Settings'
- onerror (no-speech): silently reset, don't show error

iPhone Action Button Integration

The PWA manifest and a URL parameter allow the iPhone Action Button to open Vita directly into voice mode:

```
// manifest.json — shortcuts
{
  "shortcuts": [{
    "name": "Log with voice",
    "url": "/?action=listen",
    "icons": [{ "src": "/icons/mic-192.png", "sizes": "192x192" }]
  }]
}

// app.js — on load, check for action param
const params = new URLSearchParams(window.location.search);
if (params.get('action') === 'listen') {
  voiceInput.startListening(); // auto-triggers mic
}
```

Small detail: After voice triggers and the user has a result, replace the URL with history.replaceState({}, '', '/') so refreshing doesn't re-trigger voice mode.

5.2 Photo Input (photoInput.js)

Accesses the device camera, captures a frame, encodes as base64, and passes to the Claude vision call.

Camera Access

```
// Prefer rear camera (environment)
const stream = await navigator.mediaDevices.getUserMedia({
  video: { facingMode: { ideal: 'environment' }, width: { ideal: 1280 } }
});
```

Capture & Encode

```
// Draw video frame to canvas, export as JPEG
const canvas = document.createElement('canvas');
canvas.width = video.videoWidth;
canvas.height = video.videoHeight;
canvas.getContext('2d').drawImage(video, 0, 0);
const dataUrl = canvas.toDataURL('image/jpeg', 0.85); // 0.85 quality = good balance
const base64 = dataUrl.split(',')[1];
```

Token Cost Awareness

Cost note: A 1280×960 JPEG at 0.85 quality is ~150–200KB, which costs ~800–1200 Claude input tokens. This is acceptable for MVP. In v2, resize to 800px max dimension to halve token cost. Always show a loading state after capture — vision calls take 2–4s.

Preview & Confirm Flow

9. User taps camera icon → camera UI slides up
10. Live viewfinder displayed in a <video> element
11. User taps shutter button → frame captured → stream stopped
12. Thumbnail shown with 'Add this meal?' confirm button + 'Retake' button
13. On confirm → base64 sent to claudeClient.logPhoto(base64)
14. Camera UI dismissed, processing spinner shown

5.3 Calendar Integration (calendarClient.js)

Uses Google Identity Services (GIS) for OAuth 2.0, then calls the Google Calendar API v3. No backend required — the access token lives in memory (sessionStorage for MVP).

OAuth Flow

```
// Load GIS library (add to index.html)
<script src='https://accounts.google.com/gsi/client' async></script>

// Scopes needed
const SCOPES = 'https://www.googleapis.com/auth/calendar.readonly';

// Init token client
const tokenClient = google.accounts.oauth2.initTokenClient({
  client_id: window._vitaConfig.googleClientId,
  scope: SCOPES,
  callback: (response) => {
    sessionStorage.setItem('gCalToken', response.access_token);
    calendarClient.fetchTodayEvents();
  }
});
```

Fetching Events

```
async function getTodayEvents() {
  const token = sessionStorage.getItem('gCalToken');
  if (!token) return []; // calendar not connected
  const now = new Date();
  const endOfDay = new Date(now); endOfDay.setHours(23,59,59);
  const url = `https://www.googleapis.com/calendar/v3/calendars/primary/events
    ?timeMin=${now.toISOString()}
    &timeMax=${endOfDay.toISOString()}
    &singleEvents=true&orderBy=startTime`;
  const res = await fetch(url, { headers: { Authorization: `Bearer ${token}` }});
  const data = await res.json();
  return data.items || [];
}
```

Token Expiry

Edge case: Google access tokens expire after 1 hour. Wrap every Calendar API call in a try/catch. If the response is 401, clear the stored token, show a 'Reconnect calendar' banner, and return []. Do NOT throw — failing calendar should never break meal logging.

5.4 Manual Text Input

The fallback and always-available input. A single <textarea> at the bottom of the screen that submits on Enter (desktop) or via a Send button (mobile). No special logic — text goes directly to classifyInput() then Claude.

6. Data Persistence (logStore.js)

All data is stored locally in IndexedDB using a thin wrapper. No user account or backend required for MVP. Data persists across sessions within the same browser/device.

6.1 Database Schema

Store	Key	Value shape	Purpose
profile	'profile'	{ name, age, sex, goals, dailyTargets }	User setup — written once on onboarding
foodLog	auto-increment int	{ date, time, meal, kcal, protein_g, carbs_g, fat_g, fiber_g, source, coaching_note }	Every logged meal
waterLog	auto-increment int	{ date, time, amount_l }	Water intake entries
chatHistory	auto-increment int	{ date, role, text }	Coach conversation (last 30 messages used as context)

6.2 Key Operations

```
// Get today's food log
async function getTodayLog() {
  const today = new Date().toISOString().slice(0, 10); // 'YYYY-MM-DD'
  const all = await db.getAll('foodLog');
  return all.filter(e => e.date === today);
}
```

```
// Add a meal
async function addMeal(meal) {
  const entry = {
    date: new Date().toISOString().slice(0, 10),
    time: new Date().toTimeString().slice(0, 5),
    ...meal,
    source: meal.source || 'manual' // 'voice'|'photo'|'manual'
  };
  await db.add('foodLog', entry);
  macroEngine.invalidateCache(); // force recalc on next read
  return entry;
}
```

Deletion: Add a deleteEntry(id) function from day one. Users WILL accidentally log a meal twice via voice. The UI must show an × on each log entry. Failure to build this in MVP = major UX debt.

7. Notifications (notificationManager.js + sw.js)

7.1 Overview

Vita's notifications are the most impressive feature for a demo. Unlike alarm-based reminders, they are triggered by actual calendar context: if the calendar shows a flight in 90 minutes and the user hasn't eaten, a notification fires. This requires a Service Worker.

7.2 Service Worker Registration

```
// app.js
if ('serviceWorker' in navigator) {
  const reg = await navigator.serviceWorker.register('/sw.js');
  notificationManager.init(reg);
}
```

7.3 Notification Triggers

Trigger Condition	Message Template	Priority
Flight/travel event < 90 min AND kcal today < 400	'Gate in {X} min — you've only had {food} today. Grab something now.'	HIGH
It's 12:30pm AND kcal today < 200	'You haven't eaten much yet today — quick lunch window before your afternoon.'	MED
It's 7:00pm AND kcal today < 50% of goal	'Evening check-in: you're at {kcal} kcal, {remaining} remaining. Big dinner planned?'	MED
Workout event in < 60 min AND no carbs logged in 2h	'Workout in {X} min — a quick carb snack now will help performance.'	HIGH
Water logged < 1L AND it's after 3pm	'Hydration check: only {water}L today. Aim for another {gap}L before bed.'	LOW
Client dinner event in < 3h AND today's log is heavy	'Heads up: big dinner tonight. Your lunch was {kcal} kcal — you have {budget} left.'	MED

7.4 Polling Loop

The notification manager runs a polling loop every 15 minutes using setInterval in the Service Worker's install event. It checks all trigger conditions and fires at most one notification per 2-hour window to prevent notification fatigue.

```
// sw.js
self.addEventListener('install', () => {
  setInterval(checkNotificationTriggers, 15 * 60 * 1000);
});

async function checkNotificationTriggers() {
  const state = await loadStateFromDB(); // read IndexedDB
```

```
  const lastFired = state.lastNotificationAt || 0;
  if (Date.now() - lastFired < 2 * 60 * 60 * 1000) return; // 2hr cooldown
  const trigger = evaluateTriggers(state);
  if (trigger) {
    self.registration.showNotification('Vita', {
      body: trigger.message,
      icon: '/icons/vita-192.png',
      badge: '/icons/badge-72.png',
      tag: 'vita-health',       // replaces previous notification
      renotify: true,
      data: { url: '/?action=coach' }  // open coach on tap
    });
    await saveLastNotificationTime(Date.now());
  }
}
```

Permission timing: Request notification permission during onboarding, not on first launch. Show a custom pre-prompt UI ('Vita can remind you to eat before your next flight — allow notifications?') before calling Notification.requestPermission(). This dramatically increases grant rates.

8. PWA Manifest & Install (manifest.json)

```
{
  "name": "Vita — Health Coach",
  "short_name": "Vita",
  "description": "Real-time AI personal health coach",
  "start_url": "/",
  "display": "standalone",
  "orientation": "portrait",
  "theme_color": "#1a1a2e",
  "background_color": "#1a1a2e",
  "icons": [
    { "src": "/icons/vita-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/vita-512.png", "sizes": "512x512", "type": "image/png",
"purpose": "maskable" }
  ],
  "shortcuts": [
    { "name": "Log with voice", "url": "/?action=listen", "icons": [{ "src":
"/icons/mic-192.png", "sizes": "192x192" }] },
    { "name": "Log a photo", "url": "/?action=photo", "icons": [{ "src":
"/icons/camera-192.png", "sizes": "192x192" }] }
  ],
  "categories": ["health", "fitness", "food"]
}
```

iOS install: Safari on iOS doesn't yet support manifest shortcuts or push notifications via the standard Web Push API (added in iOS 16.4+ via Home Screen installs only). Test push notifications by installing via Share → Add to Home Screen. The app must be fully installed (not just opened in Safari) for push to work on iOS.

9. UI Specification (ui.js + app.css)

9.1 Screen Hierarchy

- Onboarding (shown once, first launch): name/age/sex → goals → targets → notification permission → calendar connect → done
- Dashboard (default screen): header stats, metrics grid, food log timeline, macro bars
- Coach Drawer (bottom sheet, persistent): always accessible via bottom tab or swipe up; contains chat + quick chips
- Camera UI (modal): overlays dashboard, full-screen viewfinder
- Voice UI (overlay badge): small floating badge shows live transcript while recording

9.2 Onboarding Flow — Screen by Screen

Onboarding must be completed before any Claude calls are made. Without a profile, the context builder cannot personalize.

Screen	Fields	Notes
1. Welcome	Name (text input)	Friendly tone. 'What should Aria call you?'
2. About you	Age (number), Sex (select: male/female/prefer not to say)	Used for BMR calculation
3. Your goal	Radio: Lose weight / Maintain / Build muscle / Improve energy	Maps to calorie strategy
4. Targets	Calorie goal (auto-suggested + editable), protein/carbs/fat/fiber sliders	Auto-suggest based on profile + goal
5. Notifications	Custom pre-prompt UI → Notification.requestPermission()	Show value prop before requesting
6. Calendar	'Connect Google Calendar' button (optional, skippable)	OAuth flow; skip = calendar = empty []
7. Done	'You're all set!' summary card + 'Start logging' CTA	Navigate to Dashboard on tap

9.3 CSS Variables (Design Tokens)

```
:root {
  --color-bg-primary: #1a1a2e;       /* deep navy */
  --color-bg-secondary: #16213e;     /* panel bg */
  --color-bg-card: #0f3460;          /* card surface */
```

```
  --color-accent: #533483;            /* purple accent */
  --color-accent-light: #7f63c4;     /* hover state */
  --color-positive: #1d9e75;          /* green — good */
  --color-warning: #ef9f27;           /* amber — warn */
  --color-danger: #d85a30;            /* coral — alert */
  --color-text-primary: #f5f5f5;
  --color-text-secondary: #a0a0b0;
  --color-text-muted: #606070;
  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 20px;
  --font-sans: -apple-system, BlinkMacSystemFont, 'SF Pro Display', system-ui,
sans-serif;
}
```

9.4 Mobile-First Layout Rules

- Max content width: 430px (iPhone 15 Pro Max screen width). Center on desktop.
- Bottom tab bar: 56px tall, safe-area-inset-bottom aware. Four tabs: Dashboard, Log, Coach, Profile.
- Safe area insets: use env(safe-area-inset-bottom) for bottom padding on all fixed elements.
- Touch targets: minimum 44×44px for all interactive elements (Apple HIG).
- Coach drawer: slides up from bottom, 70vh tall, has a drag handle at top, dismisses on swipe down or backdrop tap.
- Macro bars: animated on entry using CSS @keyframes (fill from 0% to current value, 600ms ease-out).

9.5 Loading States

Every async operation must have a defined loading state. Never show a blank or frozen UI.

Operation	Loading State	Error State
Voice → Claude	Pulsing mic icon + live transcript text	'Could not log — tap to try again'
Photo → Claude	Full-screen spinner over photo thumbnail	'Could not read photo — type it instead'
Coach question	Typing indicator (3 animated dots) in chat	'Aria is unavailable — try again'
Calendar fetch	Subtle spinner in calendar section header	Silent fail — show no events, log to console
Onboarding save	Button spinner	Toast: 'Could not save — check connection'

10. Macro Engine (macroEngine.js)

The macro engine is pure local arithmetic — it never calls Claude. It reads the today log from IndexedDB and returns totals and percentage-of-goal values for rendering.

```
export function computeTodayTotals(log, goals) {
  const totals = log.reduce((acc, entry) => ({
    kcal: acc.kcal + (entry.kcal || 0),
    protein_g: acc.protein_g + (entry.protein_g || 0),
    carbs_g: acc.carbs_g + (entry.carbs_g || 0),
    fat_g: acc.fat_g + (entry.fat_g || 0),
    fiber_g: acc.fiber_g + (entry.fiber_g || 0),
  }), { kcal: 0, protein_g: 0, carbs_g: 0, fat_g: 0, fiber_g: 0 });

  return {
    totals,
    pct: {
      kcal: Math.min(100, Math.round((totals.kcal / goals.kcal) * 100)),
      protein: Math.min(100, Math.round((totals.protein_g / goals.protein_g) * 100)),
      // etc.
    },
    remaining: {
      kcal: Math.max(0, goals.kcal - totals.kcal),
    }
  };
}
```

Important: Cap all percentages at 100 using Math.min(100, ...) for UI bar widths — a 150% bar breaks the layout. Display the raw number separately so users can see they're over.

11. Build Order for Claude Code

Build in this exact sequence. Each step has a clear completion criteria before moving on.

Step	Task	Completion criteria	~Time
1	Scaffold: index.html + manifest.json + sw.js (empty) + app.css (tokens only)	App loads in browser without errors; can be installed as PWA	30 min
2	logStore.js — IndexedDB schema + all CRUD operations	Can write a meal, read today's log, delete an entry via browser console	45 min
3	contextBuilder.js + claudeClient.js (text-only, no photo yet)	Calling buildContext() returns a well-formed system prompt string	45 min
4	parseClaudeResponse.js + end-to-end meal log test	Type 'I had a banana' → JSON meal logged to IndexedDB	30 min
5	macroEngine.js + basic UI (Dashboard only, no nav)	Macros display correctly after logging a meal	45 min
6	Onboarding screens (all 7, with profile save)	Profile persisted; context builder uses real name/goals	60 min
7	voiceInput.js + iPhone Action Button URL param	Tapping mic, speaking 'I had eggs', meal logs correctly	45 min
8	photoInput.js + Claude vision call	Pointing camera at food → meal logged with macros	45 min
9	calendarClient.js + Google OAuth	Calendar events appear in context; verify in system prompt log	60 min
10	Coach drawer UI + chat history persistence	Can ask 'am I on track?' and get contextual response	45 min
11	notificationManager.js + Service Worker triggers	Simulate flight event → notification fires correctly	60 min
12	Polish: animations, loading states, error states, delete meal	All loading/error states implemented; macro bars animate	60 min
13	iOS testing: install via Safari, test push, test Action Button shortcut	Push notification works from Home Screen app; voice auto-starts	45 min

12. API Cost Management

12.1 Token Budget

Call type	Input tokens (est.)	Output tokens (est.)	Cost per call (est.)
Meal log (text)	~400 (system) + ~50 (meal desc) = ~450	~150 (JSON response)	< $0.002
Meal log (photo)	~400 (system) + ~1000 (image) = ~1400	~150 (JSON response)	< $0.005
Coach question	~400 (system) + ~30 (question) = ~430	~200 (response)	< $0.002

At 10 interactions per day per user, photo-heavy usage costs ~$0.05/user/day. Text-only is ~$0.02/user/day. This is well within a demo/MVP budget.

12.2 Optimizations for v2

- Cache calendar events for 30 minutes — don't re-fetch on every call
- Batch multiple quick text logs before sending (wait 10s after last keystroke)
- Use haiku for coach questions, sonnet only for meal parsing (haiku is 5× cheaper)
- Resize photos to 800px max before encoding — halves vision token cost
- Store last 3 coach messages in context only, not full history — reduces system prompt size

13. Edge Cases & Gotchas

These are the small details that break MVP demos. Handle all of these before calling the app done.

Scenario	Problem	Solution
User says 'I had coffee' via voice	Claude returns kcal: 5, which throws off macro display	Treat any entry < 20 kcal as a hydration item; add to water log instead, show separately
User logs same meal twice (voice misfire)	Duplicate entries in food log	Show × delete button on every log entry. Must exist on day 1.
Google token expires mid-session	Calendar fetch returns 401, crashes context builder	try/catch every calendar call; return [] on any error; show 'Reconnect' banner
Photo taken in dark room	Claude can't identify food; returns coach text instead of JSON	parseClaudeResponse fallback handles this; show 'Couldn't read photo — what did you eat?' and open text input
User in airplane mode	Claude API call fails	Queue the log entry locally with { pending: true }; retry on reconnect; show offline indicator in header
No food logged by 2pm	Context builder sends empty food log	contextBuilder must explicitly note 'No meals logged yet today' — never send an empty/missing section
User onboards without setting goals	contextBuilder.js divides by zero in pct calculation	All goal fields must have non-zero defaults. Calorie default = 2000, protein = 120g.
Voice API not available (Firefox)	SpeechRecognition is undefined	Graceful fallback: hide mic button, show text input only. Do not throw.
iOS PWA push notifications	Push only works when installed to Home Screen	Show a persistent 'Install to Home Screen for notifications' banner if running in Safari browser mode
Coach drawer keyboard overlap	iOS keyboard pushes coach drawer off screen	Set visualViewport resize listener; adjust bottom sheet translateY when keyboard appears

14. Pre-Demo Testing Checklist

Run through every item on a real iPhone (Safari, installed to Home Screen) before any demo.

Core Flow

- Complete onboarding — profile saved to IndexedDB
- Voice: tap mic, say 'I had scrambled eggs and toast' — meal logs with correct macros
- Photo: tap camera, photograph food — meal logs correctly
- Type: type 'Had a protein bar' — logs correctly
- Delete: tap × on a log entry — entry removed, macros update
- Macro bars animate correctly and cap at 100%

Calendar & Notifications

- Connect Google Calendar — today's events visible in app
- Verify calendar events appear in Claude's coaching responses
- Notification: simulate flight event < 90 min with < 400 kcal → notification fires
- Tap notification → app opens in coach mode

PWA

- Install via Safari → Add to Home Screen
- App opens full-screen (no Safari chrome)
- iPhone Action Button (if assigned to ?action=listen) auto-starts voice
- App works with 1 bar of signal (slow 3G throttle in DevTools)
- App shows offline indicator in airplane mode

Resilience

- Revoke camera permission — graceful fallback shown
- Revoke microphone permission — voice fallback shown, text input available
- Disconnect calendar — no errors, food logging still works
- Submit empty text field — nothing happens (no API call)
- Submit very long text (500+ chars) — truncated or handled gracefully

15. V2 Roadmap (Post-MVP)

The following features are explicitly out of scope for the MVP but are architecturally compatible with the current design:

Feature	Complexity	Architecture fit
Barcode scanner (camera → UPC → Open Food Facts API → log)	Medium	Add barcode detection in photoInput.js before Claude call
Apple Health / HealthKit sync (steps, HR, sleep)	High	Requires native WKWebView bridge or Capacitor wrapper
Wearable sync (Garmin, Whoop, Oura) for readiness scores	High	Add readiness object to contextBuilder; influences calorie targets
Lab result upload (PDF/photo → Claude interprets)	Medium	Add 'Labs' tab; multimodal PDF call to Claude; store structured results
Meal planning for the week (calendar-aware)	Medium	New Claude call type: 'plan'; uses full week calendar
Streaming coach responses (typewriter effect)	Low	Switch claudeClient to stream: true; render token-by-token
Multi-user / family accounts	High	Requires backend; IndexedDB partitioning or Supabase
Grocery list generation from meal plan	Low	New Claude call; outputs structured list; Web Share API

Appendix A — Environment Variables

```
# .env (never commit)
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_CLIENT_ID=...apps.googleusercontent.com
GOOGLE_API_KEY=...          # for Calendar API (non-OAuth calls)
```

Appendix B — Dependency List

Vita has zero npm runtime dependencies. All APIs are browser-native or loaded from CDN:

Dependency	Source	Purpose
Google Identity Services	https://accounts.google.com/gsi/client	OAuth 2.0 for Calendar
Google Calendar API v3	https://www.googleapis.com/calendar/v3/	Fetch calendar events
Anthropic Messages API	https://api.anthropic.com/v1/messages	All AI calls
Web Speech API	Browser built-in	Voice transcription
MediaStream API	Browser built-in	Camera access
IndexedDB	Browser built-in	Local data persistence
Service Worker API	Browser built-in	Push notifications + PWA

Appendix C — Anthropic API Headers

```
headers: {
  'Content-Type': 'application/json',
  'x-api-key': window._vitaConfig.anthropicKey,
  'anthropic-version': '2023-06-01',
  'anthropic-dangerous-direct-browser-calls': 'true'  // required for browser-side
calls
}
```

Important: The 'anthropic-dangerous-direct-browser-calls' header is required when calling the Anthropic API directly from a browser. Without it, the API returns a CORS error. This is expected for MVP/demo; in production, route calls through a server-side proxy to protect the key.

End of Vita MVP Design Document — v1.0